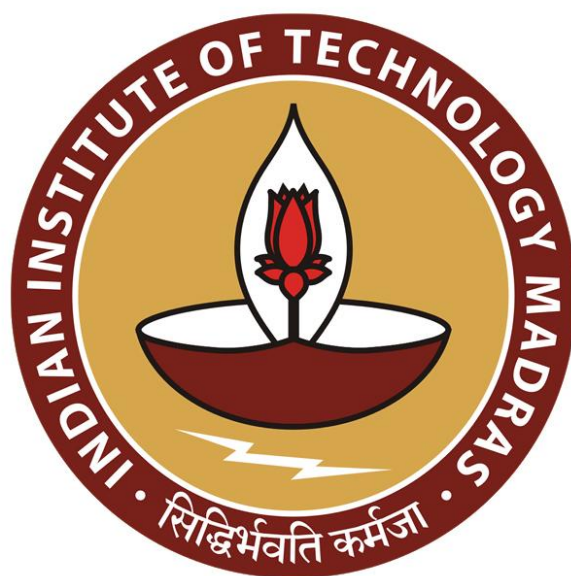




IIT Madras
ONLINE DEGREE

Programming Concepts using Java
Professor. Madhavan Mukund
Department of Computer Science
Chennai Mathematical Institute
Indian Institute of Technology, Madras
Maps

(Refer Slide Time: 00:20)

 IIT Madras
BSc Degree

Maps

- The **Collection** interface abstracts properties of grouped data
 - Arrays, lists, sets, ...
 - But **not** key-value structures like dictionaries
- Key-value structures come under the **Map** interface
 - Two type parameters
 - **K** is the type for keys
 - **V** is the type for values
 - **get(k)** fetches value for key **k**
 - **put(k,v)** updates value for key **k**

```
public interface Map<K,V> {
    V get(Object key);
    V put(K key, V Value);

    boolean containsKey(Object key);
    boolean containsValue(Object value);
    ...
}
```

Handwritten note: $e[k]$

 Madhavan Mukund

Maps

Programming Concepts using Java 2 / 6

 IIT Madras
BSc Degree

Maps

- The **Collection** interface abstracts properties of grouped data
 - Arrays, lists, sets, ...
 - But **not** key-value structures like dictionaries
- Key-value structures come under the **Map** interface
 - Two type parameters
 - **K** is the type for keys
 - **V** is the type for values
 - **get(k)** fetches value for key **k**
 - **put(k,v)** updates value for key **k**

```
public interface Map<K,V> {
    V get(Object key);
    V put(K key, V Value);

    boolean containsKey(Object key);
    boolean containsValue(Object value);
    ...
}
```

 Madhavan Mukund

Maps

Programming Concepts using Java 2 / 6

- The **Collection** interface abstracts properties of grouped data
 - Arrays, lists, sets, ...
 - But **not** key-value structures like dictionaries
- Key-value structures come under the **Map** interface
 - Two type parameters
 - **K** is the type for keys
 - **V** is the type for values
 - **get(k)** fetches value for key **k**
 - **put(k,v)** updates value for key **k**

```
public interface Map<K,V>{
    V get(Object key);
    V put(K key, V Value);
    boolean containsKey(Object key);
    boolean containsValue(Object value);
    ...
}
```

Handwritten: equals (Obj -)

Handwritten: Date:



Madhavan Mukund

Maps

Programming Concepts using Java 2/6

- The **Collection** interface abstracts properties of grouped data
 - Arrays, lists, sets, ...
 - But **not** key-value structures like dictionaries
- Key-value structures come under the **Map** interface
 - Two type parameters
 - **K** is the type for keys
 - **V** is the type for values
 - **get(k)** fetches value for key **k**
 - **put(k,v)** updates value for key **k**

```
public interface Map<K,V>{
    V get(Object key);
    V put(K key, V Value);
    boolean containsKey(Object key);
    boolean containsValue(Object value);
    ...
}
```

Handwritten: equals (Obj -)



Madhavan Mukund

Maps

Programming Concepts using Java 2/6

So, let us see now how Java deals with key value stores. So, we have looked at the collection interface. So, the collection interface captures data structures, like arrays, lists, set etc. So, these are groups of values, but they are only single values. So, in this single collection, there may or may not be a way of indexing them by position. For instance, arrays and lists will give us a positional index. So, we will have a sequence of values in a set, we have an unordered collection of values.

Now, the other important class of data structures is that called a key value pair. So, this is what Python calls a dictionary, for instance. So, here you supply not an index, but an arbitrary key and against this key, a value is stored. So, the Java name for this interface is Map. So, Map is disjoint from collection, it is a different interface. So, things which come under

collection are not under Map. So, Map and collection are two incomparable points in that tree of Java interfaces.

So, with a Map, we have two quantities, so we have a key and a value. So, this is therefore a kind of generic interface parameter is by two types. So, there is the key type. And there is the value type. And typical things that you need to do obviously in a key value store or in a dictionary is to access a value and to insert or update a value. So, these normally when we use Python, we use this square bracket notation, which is common to both lists and dictionaries.

Whereas in Java, you actually have to use these functions called get input. So, get will take a key and produce a value. So, the return type is V, the value type of string, and put will take a key and a value and insert it. So, we will come back a little bit to discuss why this has a return type V. But the other thing that you can do, of course, which you need to do often in a dictionary or in a key value store is to check whether at particular key or a particular value is present.

So, Java provides these contains key and contains values functions as part of this interface, which allows you to provide a key or a value and check whether it is there or not. So, the Boolean will obviously return true if the key is present, and not true or false if the key is absent and so on with the value.

So, before we go ahead and discuss why this put has a return type V. Another thing which is worth noting here is a discrepancy between say this put. So, put is the only one which actually seems to insist on inputs following the type that is associated with a generic Map. So, when you want to put a new pair K, V into the Map, you need to make sure that the value key that you supply is of type K, and the value that you supply is of type V.

On the other hand, when you get something, it is sufficient to provide an object. Similarly, when you search for something, I am asking, is there a key? Or is there a value present in my key value store? It is sufficient to provide an object. So, now this is a kind of design decision of Java. This is one of these many things which one can question and ask whether this is the correct way to do it.

So, you could argue that this should be a K, and this should be a V. And this should be a K. If I am going to get the value associated with a particular key, the key that I provide should be of the type that Java expects. Similarly, if I want to check whether a key is there, the key

should be of the type that Java expects. Similarly, if I want to check the value the value should V.

So, why do we have object? Well, in a sense, what are we asking? We are asking whether the key that we provide or the value that we provide is present. So, what we are really asking is that internally, what Java needs to do is compare the existing keys and values with the key or value provided using the equals operation. And equals is defined usually to also take object.

So, in a sense by providing this object as the type of the input, Java provides more flexibility. So, for instance, you might have a data structure in which equals is more generous than syntactic equality. For instance, we said that four sets, two sets would be the same if they are the same values, but they are not necessarily in the same order. Earlier we looked at data structures like date, for instance, where we said the two dates would be the same, if they actually contained the same month, day and year.

So, there might be different objects which represent the same value in some sense, which could be equated by equals, so by providing this possibility of providing an object as an argument what Java is saying is that, you provide me some representation of the key or some representation of value that you are looking for, I will tell you whether there is an equal representation present in my key value store.

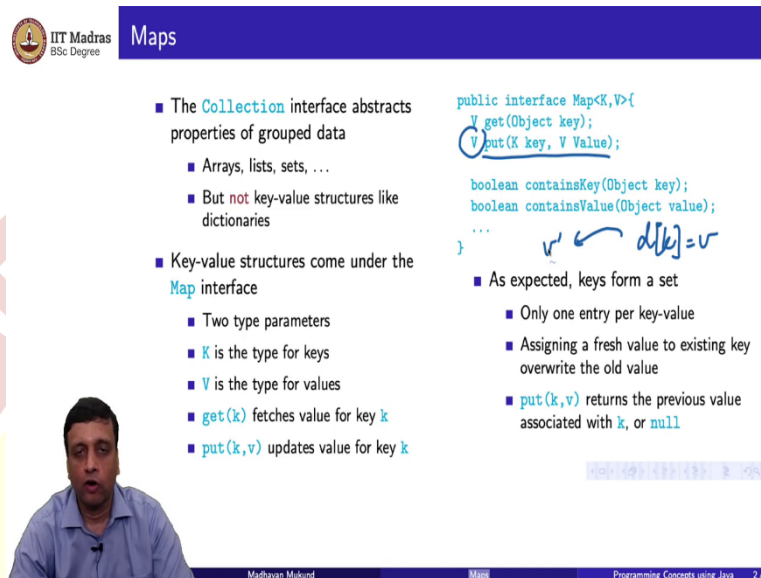
Whereas if you want to actually insert something into the key value store, then you had better be sure that the values that you are inserting are of the proper types. Now, as I said, this is a kind of debatable point, whether this generality of allowing an object is a good thing or a bad thing, it is a good thing in one sense, because you are not constraining the use of this get, and contains key and contains value.

But it is also a bad thing in the sense that now it becomes a type checking problem, because Java is not guaranteeing at compile time that the key or the value provided actually matches the type associated with the generic data structure that you are using. So, then as the programmer using or implementing this, it is your job to actually check this. So, you end up having to do some extra sanity checking of the types.

So, it is a design decision, partly this design decision is due to this issue of legacy. So, remember that generics were added later. So, many things in generics are actually designed to work with objects, because earlier that was the only way to write this kind of generic code.

So, there are various things going on behind the scenes. But it is important to note this discrepancy in this interface that only the put actually uses the types capital K and capital V. And the get contains key and contains value all allow a generic object to be passed.

(Refer Slide Time: 06:30)



Maps

- The **Collection** interface abstracts properties of grouped data
 - Arrays, lists, sets, ...
 - But **not** key-value structures like dictionaries
- Key-value structures come under the **Map** interface
 - Two type parameters
 - **K** is the type for keys
 - **V** is the type for values
 - **get(k)** fetches value for key **k**
 - **put(k,v)** updates value for key **k**

```

public interface Map<K,V>{
    V get(Object key);
    V put(K key, V Value);

    boolean containsKey(Object key);
    boolean containsValue(Object value);
    ...
}

```

As expected, keys form a set

- Only one entry per key-value
- Assigning a fresh value to existing key overwrite the old value
- **put(k,v)** returns the previous value associated with **k**, or **null**

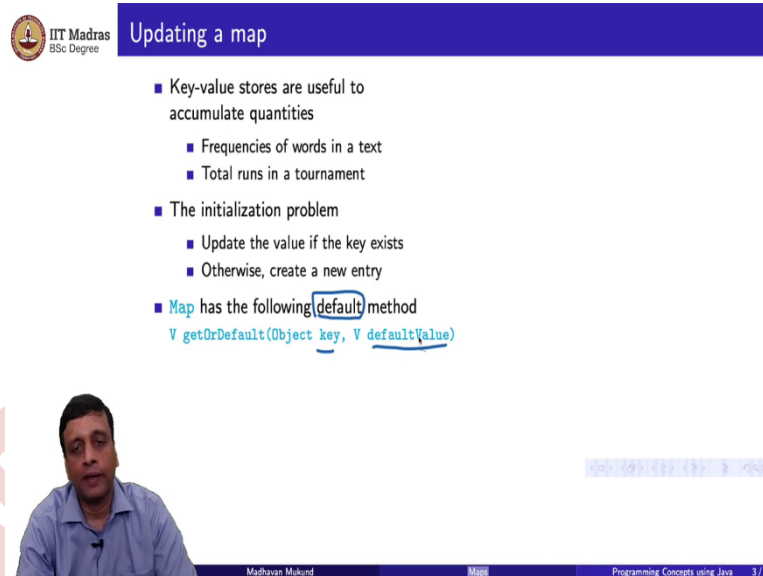
Handwritten diagram: $d[k] = v$ with an arrow pointing from v to the value position in the mapping.

So, now let us get to this issue. So, why does put return something of type v. So, we all know that in a dictionary, you cannot have two entries for the same key. So, the keys form, from an abstract point of view a set, they can only be one copy of each key, you cannot have two different keys which are equal to each other.

So, if I now update the value, at a key, the old value gets overwritten. So, this is what happens, either I create a value, we will come to that, or I overwrite the value that is there. So, when I overwrite, the old value in some sense is not available anymore. So, what Java says is that, since you are offering this through a put function, it offers you as a bonus to return to you the current value.

So, put k v updates the value associated with the k. So, it is like in a dictionary, you would say d k is equal to v, that is what the put k v does for you. But in addition, it gives you back the old v prime. So, there was already a value v prime associated with this key. So, what put does is it when you update it to a new value, it gives you back the old value. And that is why the return type of put is actually v and not something like Boolean did it work or not, or void or something like that.

(Refer Slide Time: 07:48)

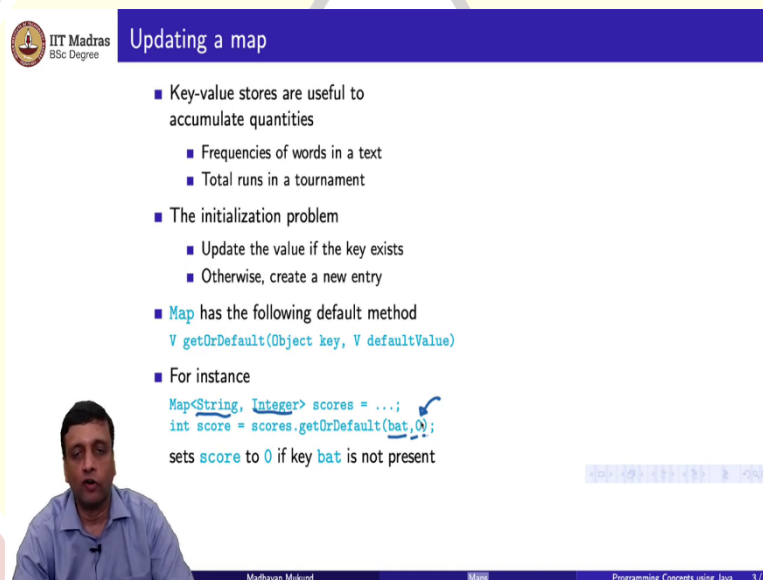


IIT Madras BSc Degree

Updating a map

- Key-value stores are useful to accumulate quantities
 - Frequencies of words in a text
 - Total runs in a tournament
- The initialization problem
 - Update the value if the key exists
 - Otherwise, create a new entry
- Map has the following default method
 - `V getOrDefault(Object key, V defaultValue)`

Madhavan Mukund Map Programming Concepts using Java 3/6



IIT Madras BSc Degree

Updating a map

- Key-value stores are useful to accumulate quantities
 - Frequencies of words in a text
 - Total runs in a tournament
- The initialization problem
 - Update the value if the key exists
 - Otherwise, create a new entry
- Map has the following default method
 - `V getOrDefault(Object key, V defaultValue)`
- For instance
 - `Map<String, Integer> scores = ...;`
 - `int score = scores.getOrDefault(bat, 0);`
 - sets `score` to 0 if key `bat` is not present

Madhavan Mukund Map Programming Concepts using Java 3/6

One of the typical uses of a key value store is to accumulate quantities. For instance, if we are running through some text, and we want to count how often each word occurs, we use the words as keys and against each key against each word we maintain account. So, every time we see that word again, we increment the count.

Similarly, we might accumulate, for instance, the runs scored by players in a tournament, so we might use the name of the player as a key. And for every score that is added to that player's record during the tournament, we will update the sum. So, we will keep adding scores and at the end that key corresponding to the players name, will record the total run scored that by that player in the tournament.

So, whenever we do this kind of accumulation, there is always an initialization problem that is the first time we want to do something differently the first time and the next time. So, the first time we encounter a word or the first time we encounter a player's runs, we need to create an entry, so we need to initialise that particular entry by creating a key for that word or that player and setting the value to be in the case of the word if it is count, we will set it to 1 if it is the case of the players score we will set it to the initial score.

Subsequently, if the key is already there, then we do not want to overwrite it what we want to do is update it. So, if it is a count, we will increment it, if it is the score, we will add the new score to the old score. So, whenever we work with these key value stores or dictionaries, we need to have a mechanism to do this.

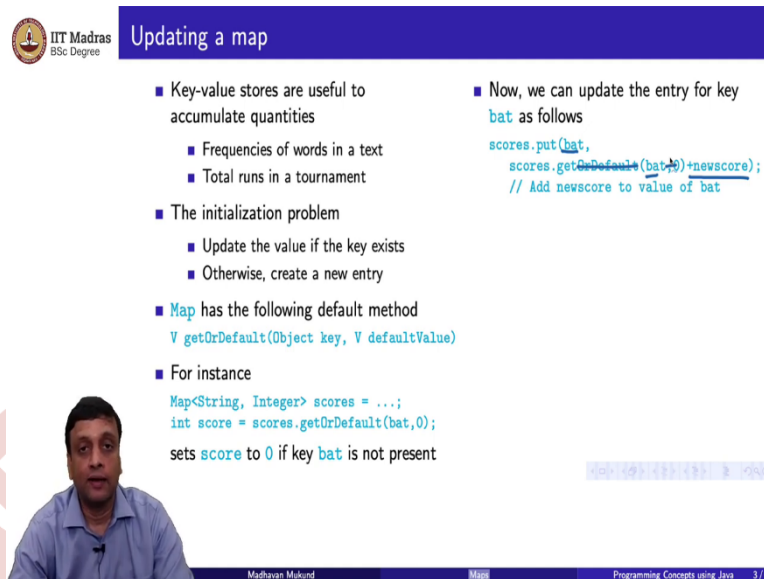
So, one mechanism that Java provides, which can be used for this is a default method. So, this method actually is given to you in the interface by default. So, it is provided to you whether you write it or not you get it. So, what this get or default method does, is it takes a key and it takes a default value as an extra parameter. So, the way you use it is like this.

So, for instance, supposing I have a Map, a dictionary, in which the keys are of type string, and the values are of type integer. So, this may be the scores of a player in a tournament. So, this string is the name of the player and the integer that is associated with that is the total runs scored by that player so far.

So, now if I pass a particular player's name, so a particular batsman's name in bat, now if there is no key, then this will give me an error, if there is no key called bat, this will give me an error. So, what get or default does is it says, I provide an extra argument 0 along with the call. So, if there is an actual value for this key bat in the dictionary or in this key value store, I will get that actual value. If there is no value there, then I will get 0.

So, 0 becomes a default return value to be returned back in case the key is not present. So, this is an extra version of get which Java provides get or default. So, you can provide a default value, if the key is there, you get the actual value, the key is not there with the default value. So, how do we use this?

(Refer Slide Time: 10:50)



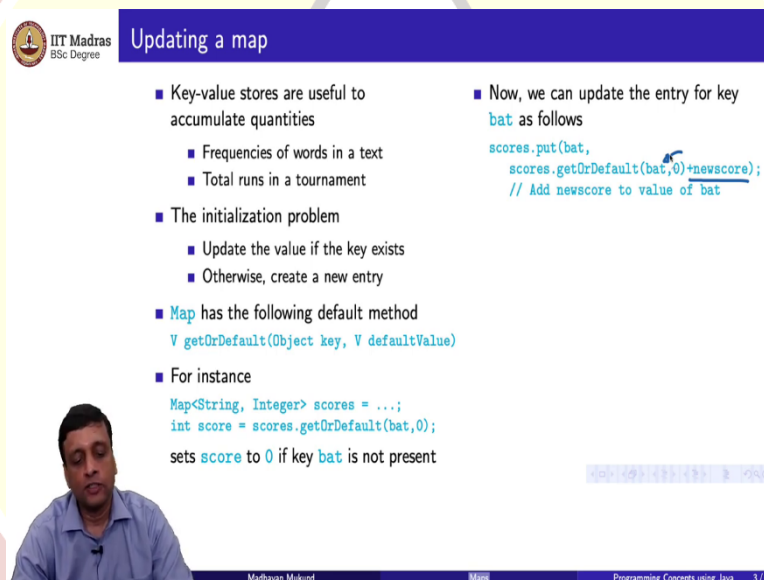
Updating a map

- Key-value stores are useful to accumulate quantities
 - Frequencies of words in a text
 - Total runs in a tournament
- The initialization problem
 - Update the value if the key exists
 - Otherwise, create a new entry
- Map has the following default method
`V getOrDefault(Object key, V defaultValue)`
- For instance

```
Map<String, Integer> scores = ...;  
int score = scores.getOrDefault(bat, 0);  
sets score to 0 if key bat is not present
```
- Now, we can update the entry for key **bat** as follows

```
scores.put(bat,  
scores.getOrDefault(bat, 0) + newscore);  
// Add newscore to value of bat
```

Madhavan Mukund | Map | Programming Concepts using Java | 3/6



Updating a map

- Key-value stores are useful to accumulate quantities
 - Frequencies of words in a text
 - Total runs in a tournament
- The initialization problem
 - Update the value if the key exists
 - Otherwise, create a new entry
- Map has the following default method
`V getOrDefault(Object key, V defaultValue)`
- For instance

```
Map<String, Integer> scores = ...;  
int score = scores.getOrDefault(bat, 0);  
sets score to 0 if key bat is not present
```
- Now, we can update the entry for key **bat** as follows

```
scores.put(bat,  
scores.getOrDefault(bat, 0) + newscore);  
// Add newscore to value of bat
```

Madhavan Mukund | Map | Programming Concepts using Java | 3/6

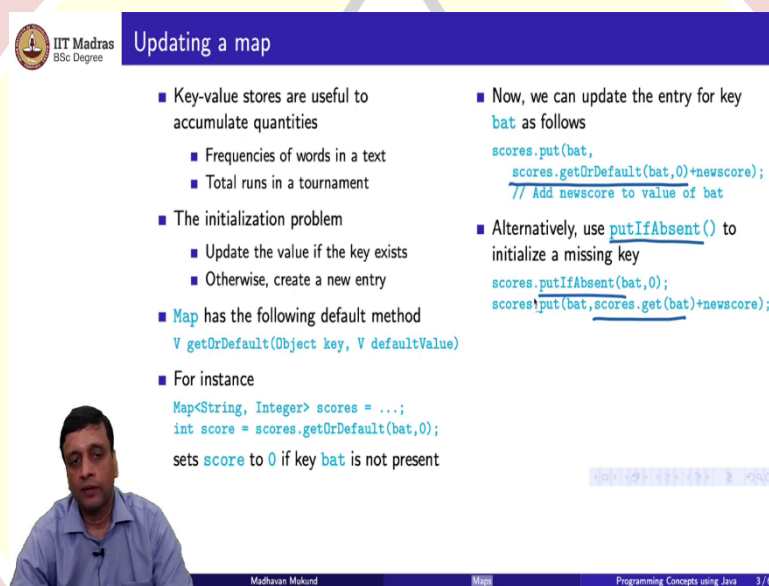
So, now what we can say is that if I want to update a value, so I get a new score, so I want to update a value. So, I want to take the existing score, and add the new score and put it back for that batter. So, there is a batter called bat against this batters' entry in the dictionary or in the key value store, there is already some total runs accumulated, I am adding one more score to that.

So, I want to take the existing value. So, I want to take the... so if I normally do it, I would just take get the current value of the score bat and add new score and put it back. So, this is a two-step process, I get the current value, add new score to it, put it back. But now because I am not sure whether that person actually has an entry or uses get or default, so what does get

or default will say? Is that, I initially get either the current score, or I get 0, which is a valid starting point for a new entry.

And then I want to initialise it, essentially, in this case, initialising it to the new score I just got, which is the same as adding 0 to that and saying put back 0 plus a new score. So, this is an indirect way of doing this in one shot. Without having to explicitly check of course, you can check as we will see, you can check the usual style would be if the key is there, then do this else do something else. But without actually having to check the key, you can use this mechanism of get or default to do this two-type update, either create a key or update the value.

(Refer Slide Time: 12:20)



Updating a map

- Key-value stores are useful to accumulate quantities
 - Frequencies of words in a text
 - Total runs in a tournament
- The initialization problem
 - Update the value if the key exists
 - Otherwise, create a new entry
- **Map** has the following default method
`V getOrDefault(Object key, V defaultValue)`
- For instance

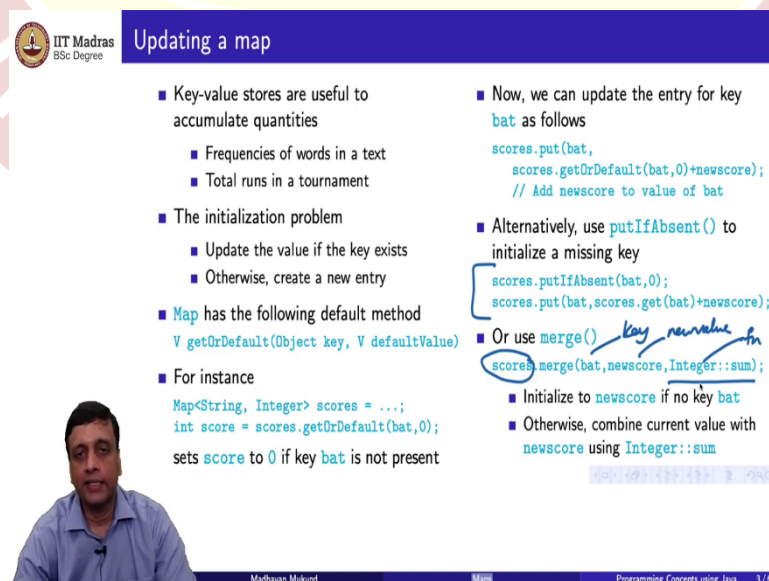
```
Map<String, Integer> scores = ...;  
int score = scores.getOrDefault(bat, 0);
```

sets **score** to 0 if key **bat** is not present
- Now, we can update the entry for key **bat** as follows

```
scores.put(bat,  
scores.getOrDefault(bat, 0) + newScore);  
// Add newScore to value of bat
```
- Alternatively, use **putIfAbsent()** to initialize a missing key

```
scores.putIfAbsent(bat, 0);  
scores.put(bat, scores.get(bat) + newScore);
```

Madhavan Mukund | Maps | Programming Concepts using Java | 3/6



Updating a map

- Key-value stores are useful to accumulate quantities
 - Frequencies of words in a text
 - Total runs in a tournament
- The initialization problem
 - Update the value if the key exists
 - Otherwise, create a new entry
- **Map** has the following default method
`V getOrDefault(Object key, V defaultValue)`
- For instance

```
Map<String, Integer> scores = ...;  
int score = scores.getOrDefault(bat, 0);
```

sets **score** to 0 if key **bat** is not present
- Now, we can update the entry for key **bat** as follows

```
scores.put(bat,  
scores.getOrDefault(bat, 0) + newScore);  
// Add newScore to value of bat
```
- Alternatively, use **putIfAbsent()** to initialize a missing key

```
scores.putIfAbsent(bat, 0);  
scores.put(bat, scores.get(bat) + newScore);
```
- Or use **merge()** *key new value for*

```
scores.merge(bat, newScore, Integer::sum);
```

 - Initialize to **newScore** if no key **bat**
 - Otherwise, combine current value with **newScore** using **Integer::sum**

Madhavan Mukund | Maps | Programming Concepts using Java | 3/6

Now, this is not the only way to do it. So, there is also a method which Java provides called `putIfAbsent`. So, as the word suggests, this creates a new entry if there is no existing value for the key. So, for instance, if I want to initialise the batsman's key to 0 provided there is no I do not want to overwrite an existing thing. But only if there is no key for this batsman, I want to create an entry with 0, I can say `putIfAbsent(bat,0)`

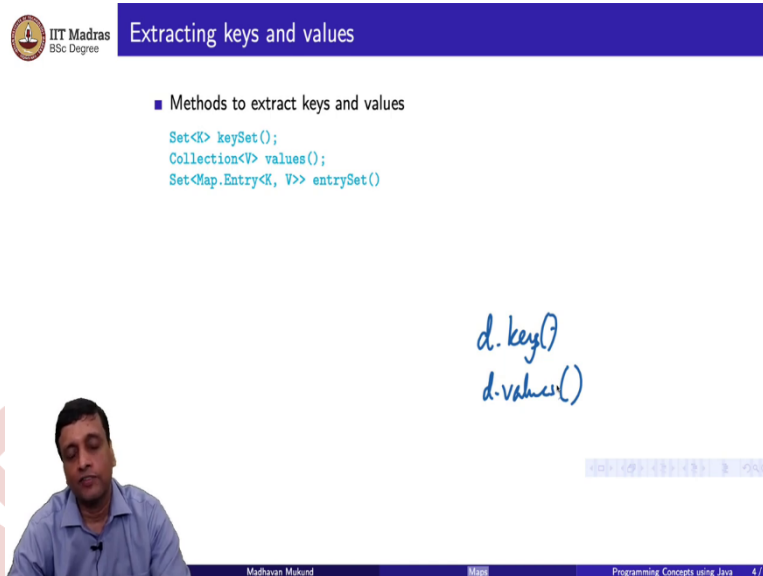
So, what this will say is if the key `bat` exists, do nothing. If the key `bat` does not exist, create a new key value pair with the key `bat` and the value 0. And now once I have done this, I am guaranteed that there is either the newly initialised 0 or there was an existing value. So, this `get` or `default` that I use there can be simplified to just a `get`. So, now I can safely look up the key `bat`, get the value by `get` add `newscore` and put it back into `bat` because of the `putIfAbsent` I have guaranteed that thing actually exists. So, this is a different way of achieving the same thing as is `get` or `default`.

And Java actually provides a third flexible way to do this something called `merge`. So, what `merge` does is it takes the name of the `Map`, and applies a `merge` to it. So, you provide a key, you provide a new value. And you provide a `merge` function, how to combine the new value with the existing thing. So, `merge` builds in this kind of initialization. So, if there is no existing key, then it works like this part. It works like this `putIfAbsent` kind of thing, it creates a key for `bat` and inserts `new score` into it.

If there is a key already, then it takes the two values, `scores`, `get back scores dot get bat and new score` and combines them using this function. So, here the function that we have provided is the `sum` function from the `integer` class. So, this is a kind of static function in the `integer` class. So, it says take the current value of the `score` associate it with `bat`, take the `newscore`, which I am providing you now, combine them using `bat`.

So, this is a combine them using `sum`, so this is a third and flexible way it says either initialise to the new value, or take the old value that already is there, the new value and combine them using the function I give you. So, Java has many ways of doing this. So, you have to choose the way that you are most comfortable with. But as we will see, you can also do an explicit thing. You can also check if it is the keys there or not.

(Refer Slide Time: 14:46)



IIT Madras
BSc Degree

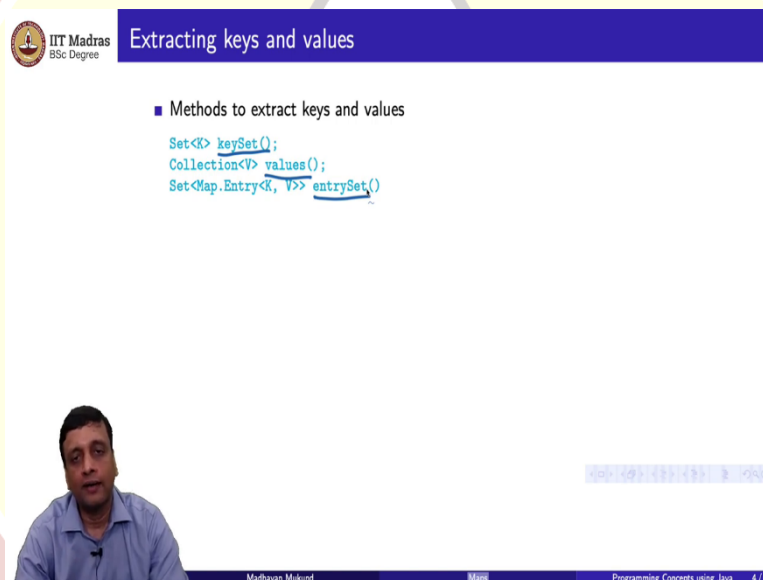
Extracting keys and values

- Methods to extract keys and values

```
Set<K> keySet();  
Collection<V> values();  
Set<Map.Entry<K, V>> entrySet()
```

*d.keys()
d.values()*

Madhavan Mukund



IIT Madras
BSc Degree

Extracting keys and values

- Methods to extract keys and values

```
Set<K> keySet();  
Collection<V> values();  
Set<Map.Entry<K, V>> entrySet()
```

Madhavan Mukund

So, how do you check if the keys there or not? Well, we need a way of extracting, as usual from a key value store the collections of keys, values and so on. So, remember in Python, you would do something like `d.keys()` or `d.values()`, so these are two built in functions which for any dictionary `d` will give you the list not quite a list, but will give you a kind of sequence of keys and values that you can run through a for loop.

So, similarly the Map interface provides these three functions. So, there is a key set, which will return the keys values, which will return the values and entry set, which will return pairs, it will return the keys and the values together. And now, these have different types and also different characteristics.

(Refer Slide Time: 15:32)



Extracting keys and values

- Methods to extract keys and values

```
Set<K> keySet();  
Collection<V> values();  
Set<Map.Entry<K, V>> entrySet();
```

- Keys form a **Set** while values form an arbitrary **Collection**

Madhavan Mukund Map Programming Concepts using Java 4/5



Extracting keys and values

- Methods to extract keys and values

```
Set<K> keySet();  
Collection<V> values();  
Set<Map.Entry<K, V>> entrySet();
```

- Keys form a **Set** while values form an arbitrary **Collection**
- Key-value pairs form a set over a special type **Map.Entry**

(k, v)

Madhavan Mukund Map Programming Concepts using Java 4/5

So, remember that keys form a set, there is only one entry per key. So, therefore the return value of key set is a set. And it is a generic set of type K, where K is the key type of the corresponding Map. So, if I have a Map with key type K and value type V, the key set function will return back a set of type K. So, set in the sense of set in the collection interface. So, it will have all the functionality associated with the set that we already saw in the collection interface.

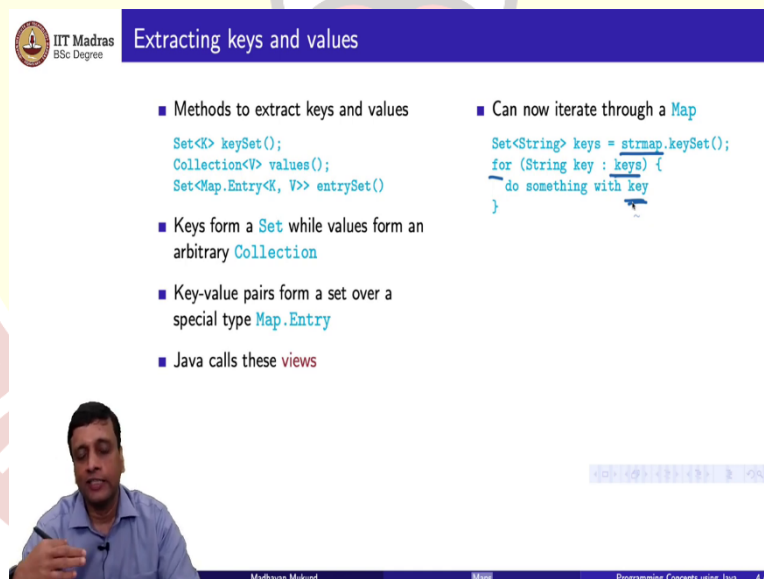
On the other hand, values have no particular structure because I could have of course, many different keys having the same value, like think of frequencies of words, many different words could appear with the same frequency, or many different batsmen could have the same

number of runs. So, values is just a collection. And it is a collection again of type V, because that is the type that we associated with the values in our key value store.

And now, the last thing is I actually get back pairs. So, these pairs are of the form K comma V, but they are not pairs in the usual sense that we use in Python, they are not just the set of pairs like this, they are actually from a type called Map dot Entry. So, the key value pairs. Now again, remember that the key is unique. So, with each key, you could have only one value.

So, if I look at the pairs key comma value, they must also be unique, I cannot have two different key value pairs which are the same, because that would imply that they both have the same key, which is not possible. So, again, this is a set and the type of value stored in the set is this complex value, which is a Map entry of type K, V. So, these are the three functions that Python with that Java gives us to actually take a Map object and produce something from the collection object.

(Refer Slide Time: 17:25)



Extracting keys and values

- Methods to extract keys and values
`Set<K> keySet();`
`Collection<V> values();`
`Set<Map.Entry<K, V>> entrySet()`
- Keys form a **Set** while values form an arbitrary **Collection**
- Key-value pairs form a set over a special type **Map.Entry**
- Java calls these **views**
- Can now iterate through a **Map**
`Set<String> keys = strmap.keySet();`
`for (String key : keys) {`
 `do something with key`
`}`

Madhavan Makund | Slide | Programming Concepts using Java 4/6

- Methods to extract keys and values

```
Set<K> keySet();
Collection<V> values();
Set<Map.Entry<K, V>> entrySet()
```

- Keys form a **Set** while values form an arbitrary **Collection**

- Key-value pairs form a set over a special type **Map.Entry**

- Java calls these **views**

if k in a.keys():

- Can now iterate through a **Map**

```
Set<String> keys = strmap.keySet();
for (String key : keys) {
    do something with key
}
```

- Use **entrySet()** to operate on key and associated value without looking up map again

```
for (Map.Entry<String, Employee> entry :
    staff.entrySet()) {
    String k = entry.getKey();
    Employee v = entry.getValue();
    do something with k, v
}
```



So, Java calls these views. So, we are used to this from a language like Python, saying that I can extract from a dictionary, the sequence of keys and the sequence of values. But here now remember, in Java, you have on one side collections, you have one side maps. So, what this is providing is a way of translating some kind of a slice of the Map, in the form of a collection either is a set of keys, or is a collection of values or a set of key value pairs.

So, as you would expect, you can now take this representation and run through your dictionary or your key value store systematically. So, for instance, if you want to go through all the entries in a key Map called a Map of whose keys are strings, so let us call it str Map, then you first extract the key set from it. And now you will use our familiar for loop to go through this set of keys, so keys now is a set.

So, we can go through it using the for loop or we can actually if you want explicitly create an iterator and use the iterator, but remember this for each loop is a kind of simplified version of the iterator. So, for each key in this set keys, we can do something with it. So, this is a simple way to do a kind of batch processing of an entire key value store.

So, typically, when we do this, we will take the key and we will go back and ask for the value associated with the key so there is a two-step process happening here we are getting the set of keys and for each key, we might again query the value. So, that is why this entry set is provided that you do not have to go back to the Map and query it you are given the key and the value together.

So, for instance, if you take entry set, you can again run the for loop, but now the for loop will be this mysterious thing. So, it will be something of type Map entry with the key type and the value type. So, this variable entry which runs through the key value pairs will have this type and I need to extract from it the two parts.

So, the projection the normal pair, you want the zeroth component and the first component so it is obtained in this case by get key and get value. So, I take this entry, it is a complex object, it has the key and the value, and we are cycling through every key and every value present in our Map. And now for each of those individual complex things, I can extract out the string part and the employee part because this was a Map in which the keys are strings and the values are employees.

So, then I can do something with the particular key and the particular value that I get and proceed. So, once I am able to extract keys values, key value pairs as collections or sets from my Map, then I can use all the iterator functions of collections in order to run through it. And in particular, going back to our previous question about checking whether something is there and then adding to it.

Well, here you can imagine, see, the thing is that if you want to check it, it is not as easy as saying in Python, if K in d.keys(), you cannot just do this as you would in Python, instead, what you have to do is you have to first extract the key set and then check whether it is in that, so every time when you update the keys, obviously, the keys change.

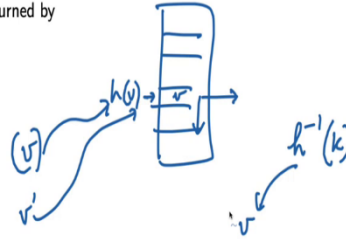
So, each time you update the keys, you have to re recreate the key set and so on, which is why it is useful to use those the other methods that we talked about like get or default or putIfAbsent and so on.

(Refer Slide Time: 20:50)

सिद्धिर्भवति कर्मजा

HashMap

- Similar to HashSet
- Use a **hash table** to store keys and values
- No fixed order over keys returned by `keySet()`



Madhavan Mukund

Map

Programming Concepts using Java 5/6

HashMap

- Similar to HashSet
- Use a **hash table** to store keys and values
- No fixed order over keys returned by `keySet()`



Madhavan Mukund

Map

Programming Concepts using Java 5/6

So, this is all remember at the level of the interface. So, these are all things that are promised to us by the Map interface. So, the Map interface comes with these get and put functions that contains functions tells us whether a key or value is present, it gives us these get or default and PutIfAbsent methods. So, all of these things are available to us because they are part of the Map interface. And then you have concrete implementations of these, just as we had collections.

So, the first concrete implementation is in the form of a hash table. So, something called a HashMap. So, remember, a hash table basically allocates something like an array and in a HashSet, we took V and we mapped it to some position h of v , and then we put the v here. So, this is what a hash table was. And now if we had a v prime, which kept the same thing, then

we had to do something about this collision thing, either we move it down or we keep some auxiliary storage or something. So, there is some kind of way of handling these collisions.

So, like a HashSet, a HashMap will do this, but now remember that because I have this possibility of collisions, I cannot say that you know, I cannot take a key and get back the value that produced it, I mean, I cannot take a position in this table and identify the key value. So, what I will have to do now is store not just the key in each entry, but I will have to store not just the value, but the key and the value.

So, that is the only logical difference between a HashMap and a hash set and a HashSet, the value itself is used as the input to the hash function, and what is stored is at that position is the value and the position is determined by v. Now, here the key is used to determine the position and new store and that position both the key and the value, because you cannot interpret that value the position to recover the key.

So, like in a HashSet, because we are kind of putting these things into the position based on the outcome of the hash function, there is no fixed order. So, this is an unordered HashMap. So, if I take out for instance, the set of keys that comes with this, it will be processed by an iterator in some order, but we cannot predict the order.

(Refer Slide Time: 23:07)

Concrete implementations of Map

HashMap

- Similar to **HashSet**
- Use a **hash table** to store keys and values
- No fixed order over keys returned by **keySet()**

LinkedHashMap

- Remembers the order in which keys were inserted
- Hash table entries are also connected as a (doubly) linked list
- Iterators over both **keySet()** and **value()** enumerate in order of insertion

TreeMap

- Similar to **TreeSet**
- Use a balanced search tree to store keys and values
- Iterator over **keySet()** will process keys in sorted order

Hand-drawn diagram of a doubly linked list with nodes containing (k1, v1), (k2, v2), and (k3, v3).

Madhavan Mukund

Maps

Programming Concepts using Java 5/5

So, if we want an order, then just like we had TreeSet set if you remember, we had TreeSet, which was in a concrete implementation of set, which underlying it use a balanced search tree. Similarly, here we can have a balanced search tree underlying a Map called a TreeMap.

And in this case, the keys actually are always available in sorted order. So, if I take out a `KeySet` from such a `Map`, then it will behave exactly like a `TreeSet`, it will give me the keys in sorted order if I run an iterator tool, so `TreeMap` corresponds to `TreeSet` `HashMap` corresponds to `HashSet`.

There is another thing which is available in both but we will first see it in the context of `HashMap`s. So, there is something called a `LinkedHashMap`. So, here I have typically remember a hash table. So, in this hash table, I have different entries I have `K1 V1`, then I might have `K2 V2` and so on and these entries could have been added in any order the positions are determined by the hash function.

Now, what a `LinkedHashMap` does is a kind of remembers it says supposing `K2 V2` came first and then `K1 V1`, then it will separately maintain a link from here to here and then `K3 V3` came next it will maintain a link, so on superimposed on top of the hash table, we have a linked list structure and in fact, this link will extract is actually a doubly linked list. So, we also have backward pointers.

So, we can traverse this linked list from beginning to end or from so we can talk about, so we can find the most recently inserted value in our `HashMap` we can find the least recently inserted value in our `HashMap` so this doubly linked list. So, what this means is now I can so this is remember this is a `HashMap` so there is no other order assumed is not a `TreeMap`.

But in this `HashMap` I am just remembering the order in which things were inserted and by doing this, I can then process these elements in that order either in the order in which they were inserted, or in the reverse order, because it is in doubly linked list, I can go backwards also. So, I can go from end to beginning a beginning to end. So, I can now process things in a in the order in which they were inserted.

And if you think about the Python dictionary, it is actually in a Python dictionary, when I say `d.keys()`. The older versions of Python did not promise you anything about the order, but the current versions of Python after 3.6 or something, promise you that `d.keys()` actually returns the keys in the same sequence in which you put the keys in.

So, really, a Python dictionary is implemented using something like a linked `HashMap`, which has a hashing part which kind of uses a hash table, but it also maintains this kind of link

structure to let you go backwards and forwards between the keys and the order in which they were inserted.

(Refer Slide Time: 25:56)

Concrete implementations of Map

HashMap

- Similar to **HashSet**
- Use a **hash table** to store keys and values
- No fixed order over keys returned by **keySet()**

TreeMap

- Similar to **TreeSet**
- Use a balanced search tree to store keys and values
- Iterator over **keySet()** will process keys in sorted order

LinkedHashMap

- Remembers the order in which keys were inserted
- Hash table entries are also connected as a (doubly) linked list
- Iterators over both **keySet()** and **value()** enumerate in order of insertion
- Can also use **access order**
 - Each **get()** or **put()** moves key-value pair to end of list
 - Process entries in **least recently used** order — scheduling, **caching**

Madhavan Mukund | Map | Programming Concepts using Java | 5/6

Concrete implementations of Map

HashMap

- Similar to **HashSet**
- Use a **hash table** to store keys and values
- No fixed order over keys returned by **keySet()**

TreeMap

- Similar to **TreeSet**
- Use a balanced search tree to store keys and values
- Iterator over **keySet()** will process keys in sorted order

LinkedHashMap

- Remembers the order in which keys were inserted
- Hash table entries are also connected as a (doubly) linked list
- Iterators over both **keySet()** and **value()** enumerate in order of insertion
- Can also use **access order**
 - Each **get()** or **put()** moves key-value pair to end of list
 - Process entries in **least recently used** order — scheduling, caching
- Similarly, **LinkedHashSet**

Madhavan Mukund | Map | Programming Concepts using Java | 5/6

So, Java goes one step further, you can also change this **LinkedHashMap** not to process in terms of the order in which the keys were inserted into the **HashMap**, but in the order in which they have been used. So, there is could be a value which was inserted a long time ago, which has not been used, but another value, which was inserted just after that has been checked again recently, or it has been updated.

So, either a **get** or a **put** against a key indicates a usage of that key. So, that key has been used recently, and it could have been a very old key it could have been put into the **Map** a long

time ago. So, the order of insertion into the Map is different from the order of access. So, we can I will not tell you the exact syntax, but we can actually set up a LinkedHashMap in which we tell it that the order you should remember is the access order.

So, what this really means is that whenever I insert whenever I get or put a key, the links are changed, so that that value goes to one end of the list, remember that the list normally goes from the earliest to the latest, now it will go from the least recently used to the most recently used. And this is important because there are many applications where we really want to know which values are recently used. So, typical examples are scheduling and caching.

So, when we are scheduling if we are maintaining a list of things that need to be done, and each of them has been allocated some amount of time, then one of the strategies which is often used is to choose for the next slice of time, the processor job which has been least recently scheduled. So, if you can maintain some kind of fairness across this, another situation is caching.

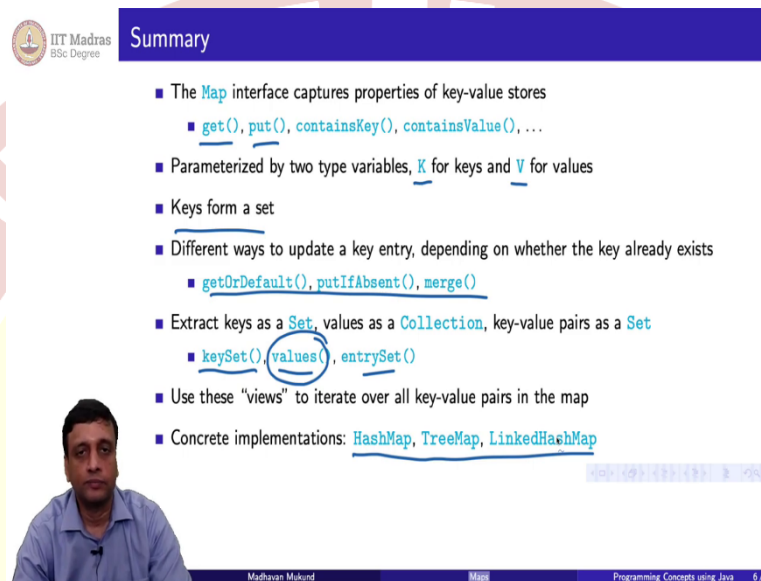
So, caching refers to the idea that if you are running out of memory, you can put some of your so the data from your memory can go onto the disk or you might have in certain cases, you might have some superfast memory inside the processor and you might want to keep a subset of things there. So, which are the things that you need to keep in fast memory and which are the things can go to slow memory.

So, typically when you run out of space, and you want to bring something in from the slow memory to the fast memory something has to go out. And usually there what goes out is something which you have not used for a long time. So, imagine that you have a large collection of books in your house. So, you have a bookshelf in your bedroom, which can store all the books and in your living room, you have a small bookshelf, where you can keep 10 books.

So, these 10 books now, at some point, you want to read a book from the bedroom and you want to keep it outside. So, one of these 10 books has to go back to make space for it. So, which book will you put back the one that you have not read for a long time. So, that is this least recently used policy and that can be implemented using this access order in a LinkedHashMap.

So, just like there is a LinkedHashMap. So, just like HashMap is like a HashSet and TreeMap is like a TreeSet, there is also a corresponding LinkedHashMap. So, even in a set, we can have the elements in the hash table linked either in the order in which they were inserted, or in the order in which they have been accessed. So, it does not have to be only for a key value store even for a set we can use the same data structure.

(Refer Slide Time: 29:03)



Summary

- The `Map` interface captures properties of key-value stores
 - `get()`, `put()`, `containsKey()`, `containsValue()`, ...
- Parameterized by two type variables, `K` for keys and `V` for values
- Keys form a set
- Different ways to update a key entry, depending on whether the key already exists
 - `getOrDefault()`, `putIfAbsent()`, `merge()`
- Extract keys as a `Set`, values as a `Collection`, key-value pairs as a `Set`
 - `keySet()`, `values()`, `entrySet()`
- Use these "views" to iterate over all key-value pairs in the map
- Concrete implementations: `HashMap`, `TreeMap`, `LinkedHashMap`

Madhavan Mukund | Maps | Programming Concepts using Java | 6/6

So, to summarise, we have two separate interfaces in Java. We have the Collection interface, which talks about things which are not key value stores and we have the Map interface which captures the properties of key value stores. So, the key value store requires us to associate a key with a value, so get will tell us the current value associated with the key, put will insert a new entry for a key and contains key and contains value will tell us whether the given key is present or not and a given value is present or not.

So, here it is a generic data structure in there, I mean it is a generic interface, which has two parameters in terms of types, one for the type of keys and one for the type of values because they can be from different universes I can have a different type of key and a different type of value. But like in every key value store like in a dictionary thing whenever you see key value store, think of a Python dictionary, you can only have one copy per key, so the keys will almost set.

Then one of the fundamental problems that arises when we update values and the key value store is to decide whether or not it is the first time we are assigning it, whether we are

updating it. So, we have these different mechanisms, which allow us without explicitly querying, whether it is present or not. We have different mechanisms to actually get hold of these values and update them, so we have this get or default, putIfAbsent and merge.

And then, as in any reasonable key value store, we can extract out the collection of keys and values. And in this particular case, even the key value pairs, using this key set values and entry set, we will remember values will give you an arbitrary collection, the other two things will give you specifically a set, because there can only be one key, one copy of every key and they can only be one copy of every key value pair.

So, these are called views. And using these views, we can run this for each loop, or run an iterator to run over a data structure. And then we saw that there are three concrete implementations which are commonly available in Java for this for this key value store. There is a HashMap, which is like a hash set a TreeMap, which is like a TreeSet.

So, a HashMap uses a hash table. A TreeMap uses the binary kind of a search tree in order to maintain it. And a linked HashMap is a augmented HashMap in which we additionally maintain a linked list structure between the entries in the hash table so that we can keep track of the order in which they were inserted. Or we can keep track of the order in which they were updated. So, we can talk about the most recently used and the least recently used.